

Gate Engine Implementation Guide

Step 1: Add Gate Engine Cell (After ML Bridge Cell)

```
python
```

```
# === Gate Engine Implementation Cell ===  
# Add this as cell 12 in your notebook (after the ML bridge, before UI)
```

```
from dataclasses import dataclass  
from typing import Dict, Any, List, Optional  
from datetime import datetime, timezone  
import json
```

```
@dataclass
```

```
class GateStatus:  
    category: str  
    gate_id: str  
    status: str # "open", "blocked", "pending"  
    readiness_score: float # 0.0-1.0  
    blockers: List[str]  
    eta_minutes: Optional[int]  
    actions_required: List[str]
```

```
class GateEngine:  
    def __init__(self):  
        # Your operational constraints (tune these based on your hospital)  
        self.constraints = {  
            "bed_manager_cutoff_hour": 16, # 4pm  
            "night_shift_start_hour": 22, # 10pm  
            "troponin_delta_hours": 3,  
            "mandatory_hold_conditions": ["abdominal", "abdom"],  
            "ct_queue_threshold_minutes": 120,  
            "lab_turnaround_threshold_minutes": 90,  
        }
```

```
# [Include the full GateEngine implementation from the first artifact]
```

```
# Global instance
```

```
gate_engine = GateEngine()  
print("Gate engine loaded with operational constraints")
```

Step 2: Modify Your Existing score_and_log Function

python

=== Enhanced Scoring Function ===

Replace your existing score_and_log function

```
def score_and_gate(row: dict, calculators: dict = None, patient_context: dict = None):
```

```
    """
```

```
    Enhanced scoring: ML + Gate Engine
```

```
    Keeps your existing ML for comparison, adds actionable gate assessment
```

```
    """
```

```
    # Your existing ML prediction (for comparison)
```

```
    try:
```

```
        ml_res = predict_one(row)
```

```
        _append_event({"type": "ml_score", "res": ml_res, "keys": list(row.keys())})
```

```
        ml_useful = ml_res['p'] > 0.01 # Your current model: always False
```

```
    except Exception as e:
```

```
        ml_res = {"p": 0.0, "y": 0, "thr": 0.999, "error": str(e)}
```

```
        ml_useful = False
```

```
    # Gate engine assessment (primary decision maker)
```

```
    gate_result = gate_engine.compute_overall_readiness(row, calculators, patient_context)
```

```
    # Enhanced event logging
```

```
    _append_event({
```

```
        "type": "enhanced_assessment",
```

```
        "patient_id": row.get("Fall-ID", "unknown"),
```

```
        "ml_result": ml_res,
```

```
        "ml_useful": ml_useful,
```

```
        "gate_priority": gate_result["priority"],
```

```
        "gate_readiness": gate_result["overall_readiness"],
```

```
        "next_action": gate_result["next_action"],
```

```
        "gate_details": {k: {"status": v.status, "score": v.readiness_score}
```

```
            for k, v in gate_result["gate_details"].items()})
```

```
    })
```

```
    # Display results
```

```
    print(f"ML risk p={ml_res['p']:.3f} → {'ALERT' if ml_res['y'] else 'ok'} (thr={ml_res['thr']:.3f}) {' USELESS' if not ml_res['p'] else 'OK'}")
```

```
    print(f"Gate priority: {gate_result['priority']}/10 | Readiness: {gate_result['overall_readiness']:.2f} | {gate_result['next_action']}")
```

```
    print(f"Next action: {gate_result['next_action']}")
```

```
    if gate_result["eta_minutes"]:
```

```
        hours = gate_result["eta_minutes"] // 60
```

```
        mins = gate_result["eta_minutes"] % 60
```

```
        eta_str = f"{hours}h {mins}m" if hours > 0 else f"{mins}m"
```

```
        print(f"ETA: {eta_str}")
```

```
    return {
```

```
        "gate_result": gate_result,
```

```
"ml_result": ml_res,  
"gate_result": gate_result,  
"primary_decision": gate_result, # Use gate engine for operational decisions  
"ml_useful": ml_useful  
}
```

```
# Backward compatibility alias
```

```
score_and_log = score_and_gate
```

```
print("Enhanced scoring function ready - combines ML + Gate Engine")
```

Step 3: Enhance Your Phase-2 Calculator Integration

python

```
# === Phase-2 Calculator + Gate Engine Integration ===  
# Modify your existing phase-2 calculator button handler
```

```
def on_compute(_): # Your existing button handler  
    with out:  
        out.clear_output()  
        try:  
            # Parse inputs (your existing code)  
            vitals = json.loads(vitals_in.value or "{}")  
            labs = json.loads(labs_in.value or "{}")  
            ctx = json.loads(ctx_in.value or "{}")  
  
            # Add HEART components (your existing code)  
            vitals["heart_history"] = int(heart_hist.value)  
            vitals["heart_ecg"] = int(heart_ecg.value)  
            vitals["heart_risk"] = int(heart_risk.value)  
  
            # Your existing calculator bundle  
            bundle = phase2_bundle(vitals, labs, ctx)  
  
            # NEW: Gate engine integration  
            # Convert bundle to gate engine inputs  
            features = {  
                # Map from your UI to 16 ML features  
                'Tag': 1, # Assume high acuity if using calculator  
                't_min': ctx.get('t_min', 120),  
                'Triage': ctx.get('triage', 2),  
                'Leitsymptom': ctx.get('chief_complaint', 'chest_pain'),  
                'HF': vitals.get('hr', 80),  
                'MAP': vitals.get('map', 80),  
                'ICU_Kap': ctx.get('icu_capacity', 5),  
                'Kap_veraltet': False,  
                't_norm': 1.0,  
                'hat_Labor': bool(labs),  
                'Labor_ausstehend': labs.get('pending', False),  
                'hat_Roentgen': ctx.get('cxr_done', True),  
                'Roentgen_ausstehend': ctx.get('cxr_pending', False),  
                'hat_CT': ctx.get('ct_done', False),  
                'CT_ausstehend': ctx.get('ct_pending', False),  
                'naechste_Aktion': ctx.get('next_action', 'unknown'),  
                'age': vitals.get('age', 65)  
            }  
        
```

```
# Calculator results for gate engine
```

```

calculators = {
    'heart': bundle['scores']['heart'],
    'qsofa': bundle['scores']['qsofa'],
    'troponin_flag': bundle['labs'].get('trop_flag', False),
    'requires_pregnancy_test': ctx.get('pregnant') is None and vitals.get('age', 100) < 50
}

# Patient context
patient_context = {
    'age': vitals.get('age', 65),
    'has_gp': ctx.get('has_gp', True),
    'nursing_home_placement_needed': vitals.get('age', 65) >= 85,
    'needs_home_support': ctx.get('needs_home_support', False),
    'home_support_arranged': ctx.get('home_support_arranged', True)
}

# Run enhanced assessment
assessment = score_and_gate(features, calculators, patient_context)

# Display existing calculator results
print("\n".join(bundle["one_liners"]))

# Display gate engine results
print("\n" + "="*50)
print("OPERATIONAL ASSESSMENT")
print("="*50)
gate_details = assessment["gate_result"]["gate_details"]
for category, gate in gate_details.items():
    status_emoji = {"open": "🟢", "pending": "🟡", "blocked": "🔴"}.get(gate.status, " ? ")
    print(f"{status_emoji} {category.replace('_', ' ').title(): {gate.status}")
    if gate.blockers:
        print(f"   Blockers: {' '.join(gate.blockers)}")
    if gate.actions_required:
        print(f"   Actions: {' '.join(gate.actions_required[:2])}") # Show first 2

print(f"\nOPERATIONAL PRIORITY: {assessment['gate_result']['priority']/10}")
print(f"NEXT STEP: {assessment['gate_result']['next_action']}")

print("\nLogged to:", EVENT_LOG_PATH)

except Exception as e:
    print("[error]", e)

```

Step 4: DataFrame Integration (Replace Gate_pos)

python

```
# === DataFrame Enhancement Cell ===
```

```
# Add this to replace any DataFrame operations that used Gate_pos
```

```
def replace_gate_pos_in_dataframes():
```

```
    """
```

```
    Find DataFrames with Gate_pos column and replace with gate engine outputs
```

```
    """
```

```
    import pandas as pd
```

```
    # Find DataFrames in global scope
```

```
    dataframes_updated = 0
```

```
    for name, obj in globals().items():
```

```
        if isinstance(obj, pd.DataFrame) and 'Gate_pos' in obj.columns:
```

```
            print(f"Updating DataFrame '{name}' with gate engine...")
```

```
            enhanced_df = enhance_dataframe_with_gates(obj)
```

```
            # Replace the global DataFrame
```

```
            globals()[name] = enhanced_df
```

```
            dataframes_updated += 1
```

```
            print(f"✅ Replaced Gate_pos with: priority, gates, next_action, readiness_score, eta_minutes")
```

```
    if dataframes_updated == 0:
```

```
        print("No DataFrames with Gate_pos found")
```

```
    else:
```

```
        print(f"Updated {dataframes_updated} DataFrames")
```

```
def enhance_dataframe_with_gates(df: pd.DataFrame) -> pd.DataFrame:
```

```
    """
```

```
    Add gate engine columns to DataFrame, remove Gate_pos
```

```
    """
```

```
    results = []
```

```
    for idx, row in df.iterrows():
```

```
        # Convert DataFrame row to feature dict
```

```
        features = {
```

```
            col: row[col] for col in gate_engine.feature_names if col in df.columns
```

```
        }
```

```
        # Fill missing features with defaults
```

```
        feature_defaults = {
```

```
            'Tag': 2, 't_min': 120, 'Triage': 3, 'Leitsymptom': 'unknown',
```

```
            'HFL': 20, 'IMADL': 20, 'QOL_Kent': 5, 'Ken_versus_koth': False
```

```

'HF': 80, 'MAP': 80, 'ICU_Kap': 5, 'Kap_veraltet': False,
't_norm': 1.0, 'hat_Labor': True, 'Labor_ausstehend': False,
'hat_Roentgen': True, 'Roentgen_ausstehend': False,
'hat_CT': False, 'CT_ausstehend': False, 'naechste_Aktion': 'unknown'
}

```

```

for key, default in feature_defaults.items():
    if key not in features:
        features[key] = default

```

Basic calculators (enhance based on your data)

```

calculators = {
    'heart': 3, # Default moderate risk
    'qsofa': 0, # Default no sepsis
    'troponin_flag': False
}

```

Patient context

```

patient_context = {
    'age': features.get('age', row.get('age', 65)),
    'has_gp': True,
    'nursing_home_placement_needed': features.get('age', 65) >= 85
}

```

Compute gate assessment

```

gate_result = gate_engine.compute_overall_readiness(features, calculators, patient_context)
results.append(gate_result)

```

Create enhanced DataFrame

```

df_enhanced = df.copy()

```

Remove Gate_pos if present

```

if 'Gate_pos' in df_enhanced.columns:
    df_enhanced = df_enhanced.drop('Gate_pos', axis=1)

```

Add gate engine columns

```

df_enhanced['priority'] = [r['priority'] for r in results]
df_enhanced['gates'] = [r['gates'] for r in results]
df_enhanced['next_action'] = [r['next_action'] for r in results]
df_enhanced['readiness_score'] = [r['overall_readiness'] for r in results]
df_enhanced['eta_minutes'] = [r['eta_minutes'] for r in results]

```

```

return df_enhanced

```

Run the replacement

```

if CONFIG.get("RUN_PIPELINE"):
    replace_gate_pos_in_dataframes()

```

Step 5: UI Updates (Optional - Enhance Your Existing Widgets)

python

=== Enhanced UI Display ===

Add this to your existing UI cells to show gate engine outputs

```
if CONFIG.get("RUN_UI"):
```

```
    try:
```

```
        import ipywidgets as W
```

```
        # Gate Engine Status Widget
```

```
        gate_status_output = W.Output()
```

```
    def update_gate_display(assessment_result):
```

```
        """Update UI with gate engine results"""
```

```
        with gate_status_output:
```

```
            gate_status_output.clear_output()
```

```
            if 'gate_result' in assessment_result:
```

```
                gate_result = assessment_result['gate_result']
```

```
            # Priority display
```

```
            priority = gate_result['priority']
```

```
            priority_color = "red" if priority >= 7 else ("orange" if priority >= 4 else "green")
```

```
            display(W.HTML(f"""
```

```
                <div style="border: 2px solid {priority_color}; padding: 10px; margin: 5px; border-radius: 5px;">
```

```
                    <h4>Operational Assessment</h4>
```

```
                    <p><strong>Priority:</strong> {priority}/10</p>
```

```
                    <p><strong>Status:</strong> {gate_result['gates']}</p>
```

```
                    <p><strong>Next Action:</strong> {gate_result['next_action']}</p>
```

```
                    <p><strong>Readiness:</strong> {gate_result['overall_readiness']:.1%}</p>
```

```
                </div>
```

```
            """))
```

```
        # Add to your existing UI
```

```
        display(W.VBox([
```

```
            W.HTML("<b>Gate Engine Status</b>"),
```

```
            gate_status_output
```

```
        ]))
```

```
except Exception as e:
```

```
    print("Enhanced UI unavailable:", e)
```


Step 6: Testing & Validation

python

=== Testing Cell ===

Validate gate engine with known scenarios

def test_gate_engine():

"""Test gate engine with realistic scenarios"""

test_scenarios = [

{

"name": "Ready for discharge",

"features": {

'Tag': 3, 't_min': 240, 'Triage': 4, 'Leitsymptom': 'minor_complaint',

'HF': 70, 'MAP': 85, 'ICU_Kap': 8, 'hat_Labor': True, 'Labor_ausstehend': False,

'hat_CT': False, 'CT_ausstehend': False, 'naechste_Aktion': 'discharge'

},

"expected_priority": "low (1-3)"

},

{

"name": "Blocked by CT queue",

"features": {

'Tag': 2, 't_min': 90, 'Triage': 2, 'Leitsymptom': 'abdominal_pain',

'HF': 90, 'MAP': 80, 'ICU_Kap': 5, 'hat_Labor': True, 'Labor_ausstehend': False,

'hat_CT': False, 'CT_ausstehend': True, 'naechste_Aktion': 'surgery_consult'

},

"expected_priority": "high (7-9)"

}

]

print("Testing Gate Engine...")

for scenario in test_scenarios:

print(f"\n🔪 Scenario: {scenario['name']}")

result = score_and_gate(scenario['features'])

priority = result['gate_result']['priority']

print(f" Expected: {scenario['expected_priority']}")

print(f" Actual: {priority}/10")

print(f" Status: {✅ PASS' if 1 <= priority <= 10 else '❌ FAIL'}")

if CONFIG.get("RUN_PIPELINE"):

test_gate_engine()

Implementation Timeline

Week 1: Core Implementation

- ☐ Add GateEngine class (Step 1)
- ☐ Modify scoring function (Step 2)
- ☐ Test with sample data
- ☐ Verify event logging works

Week 2: Integration

- ☐ Integrate with phase-2 calculators (Step 3)
- ☐ Update DataFrame processing (Step 4)
- ☐ Test with real patient data
- ☐ Validate against operational knowledge

Week 3: UI & Refinement

- ☐ Enhance UI displays (Step 5)
- ☐ Add comprehensive testing (Step 6)
- ☐ Fine-tune operational constraints
- ☐ User acceptance testing

Week 4: Deployment

- ☐ Run parallel with existing system
- ☐ Compare gate engine vs ML predictions
- ☐ Phase out Gate_pos column
- ☐ Full operational deployment

Key Success Metrics

1. **Actionability:** Every high-priority patient has specific next steps
2. **Accuracy:** Gate engine correctly identifies operational bottlenecks
3. **Adoption:** Clinical staff find gate outputs more useful than ML predictions
4. **Integration:** Seamless with existing WorkflowState and calculator infrastructure
5. **Performance:** Real-time assessment without system slowdown

This implementation keeps your existing infrastructure while replacing the broken ML prediction with actionable operational intelligence.